

Kick Start 2020 - Round E

Analysis: Golden Stone

The given set of junctions and streets can be considered as an undirected graph. The problem statement guarantees that the graph is connected.

We can consider all possible combinations of junctions and stones, and try to figure out the optimal cost of making that stone at that junction. Let's define the cost of gathering stone c at junction j as $\text{Cost}_{j, c}$. Then the solution would be the minimum of $\text{Cost}_{\text{junction}, \text{golden}}$ over all the junctions.

Test set 1

We can consider a new graph, where each {junction, stone type} combination is a vertex, and for each street $\{u, v\}$ in the given input, there will be $\mathbf{S}$ new edges $\{(u, c), (v, c)\}$, one for each stone type c . Then we can run a variation of [Bellman-Ford algorithm](#) on this graph to fill the two dimensional cost table as mentioned above.

Firstly, for all the stone types that are directly available at some junctions (as given in the input), the $\text{Cost}_{\text{junction}, \text{stone_type}}$ would be 0. All other combinations of junction and stone type will have initial cost set to infinity.

Then we can try to relax the cost as in the classic Bellman-Ford algorithm, with some modification, that, in addition to relaxing edges, we also try to reduce cost by applying recipes on the stones that are available at the same junction at some point.

For each junction u , we can relax the cost table as follows:

- For each edge $\{v, u\}$ in the input graph, and stone type c , try to reduce the $\text{Cost}_{u, c}$ by carrying one type c stone from v to u .
- Try to apply each recipe at the junction u to see if we can improve the $\text{Cost}_{u, c}$ where c is the stone produced by the recipe.

We must repeat the relaxation steps mentioned above for each junction as long as there are any improvements made to the cost values. It can be proved that no more than $\mathbf{N} \times \mathbf{S}$ iterations are required.

For the first type of relaxation, each {edge, stone type} combination is considered only once, and then the whole process is repeated $O(\mathbf{N} \times \mathbf{S})$. So, the overall complexity is $O(\mathbf{N} \times \mathbf{M} \times \mathbf{S}^2)$.

For the second type of relaxation, the process takes $O(\mathbf{R})$ time per junction, and the whole relaxation process will repeat $O(\mathbf{N} \times \mathbf{S})$ times per junction. So the complexity is $O(\mathbf{N}^2 \times \mathbf{S} \times \mathbf{R})$.

So, the total complexity of this approach is $O(\mathbf{N} \times \mathbf{S} \times ((\mathbf{M} \times \mathbf{S}) + (\mathbf{N} \times \mathbf{R})))$.

Test set 2

Filling up the cost table can also be achieved with [Dijkstra's algorithm](#). The cost table can be initialized in the same approach used in test set 1. Then the vertices with cost 0 are put into a minimum priority queue with the calculated cost as the key.

At each step of trying to reduce cost for a vertex $\{u, c\}$ from the queue, we can do the following:

- For each edge $\{u, v\}$, try to reduce the $\text{Cost}_{v, c}$ by carrying one type c stone from u to v .
- Try to reduce the $\text{Cost}_{u, \text{stone_type}}$ by applying recipes where c is an ingredient.

In this approach, there are $\mathbf{N} \times \mathbf{S}$ vertices. For each edge in the input graph there will be corresponding $\mathbf{S}$ edges, so in total $\mathbf{M} \times \mathbf{S}$ edges.

The relaxation of the first type follows classic Dijkstra's algorithm's style, so the complexity is $O((\mathbf{N} \times \mathbf{S} + \mathbf{M} \times \mathbf{S}) \times \log(\mathbf{N} \times \mathbf{S}))$.

For the relaxation of the second type for each vertex $\{u, c\}$, we need to try only the recipes where c is an ingredient. So in total, for each junction, a recipe will be applied once for each of its ingredients. Each recipe has at most 3 ingredients. Which gives us additional $O(\mathbf{R} \times \mathbf{N} \times \log(\mathbf{N} \times \mathbf{S}))$ complexity. So the total time complexity of this approach is $O((\mathbf{N} \times \mathbf{S} + \mathbf{M} \times \mathbf{S} + \mathbf{R} \times \mathbf{N}) \times \log(\mathbf{N} \times \mathbf{S}))$ which is sufficient for test set 2.

Analysis: High Buildings

Let the heights of buildings be $H_1, H_2, H_3, \dots, H_N$. If we look from the left side, it is possible to view a building i , iff $H_i \geq H_j$ for all $1 \leq j \leq i$. Similarly, if we look from the right side, it is possible to view a building i , iff $H_i \geq H_j$ for all $i \leq j \leq N$. It is possible to view a building i from both the sides if $H_i \geq H_j$ for all $i \leq j \leq N$ and $H_i \geq H_k$ for all $1 \leq k \leq i$. This means that $H_i \geq H_j$ for all $1 \leq j \leq N$. Hence, only the buildings which have the maximum height would be visible from both sides.

The answer for $N = 1$ case is trivial. Consider the case where $A + B - C > N$. A buildings are seen from the left side, B buildings are visible from the right side and C buildings are visible from both sides. This means that $A + B - C$ buildings will be visible from at least one of the sides. Hence, there are more than N buildings which are visible from at least one side, which is not possible. Thus, answer is IMPOSSIBLE in case $A + B - C > N$. The rest of the analysis assumes $N > 1$ and $A + B - C \leq N$.

Test Set 1

The height of each building is an integer between 1 to N , inclusive. Thus, each building can have at most N distinct values. So, if we consider all possible heights of the given buildings, we get N^N different combinations. Now, we need to check if a possible set of heights satisfy the given condition of A , B and C .

To check how many buildings are visible from the left side, we start iterating from the left, and maintain a `prefix_max` variable that indicates what is the maximum height of a building which are on the left side of the current building. Formally, for building i , `prefix_max` would indicate $\max(H_j)$ for $1 \leq j < i$. And then we update the `prefix_max` variable when we move onto the next index. This can be done in $O(N)$. Similarly, we can find how many buildings are visible from the right side by iterating from the right side, and maintaining a suffix maximum variable. This can be done in $O(N)$. To count the number of buildings, which are visible from both sides, we need to find the maximum height and count the number of buildings with such maximum height. This can be done in $O(N)$ too. If for a given set of heights, we get buildings visible from left side equal to A , buildings visible from right side equal to B and buildings visible from both the sides equal to C , we have found our answer. If there is no such set of heights that satisfy this condition, then we say that it is not possible.

A given set of height can be checked in $O(N)$ time. There are N^N different combinations. Thus the overall complexity of the solution is $O(N^N \times N)$, which runs under the time limit for $N \leq 5$.

Test Set 2

In this test set, we cannot generate all possible set of heights as we have $N \leq 100$. This can be solved by splitting into some cases:

- Case 1: Let us first assume that $C > 1$. Consider any 2 values P and Q such that $1 \leq P < Q \leq N$. In this case, we can put $A - C$ buildings with height P on the left and $B - C$ buildings with height P on the right and C buildings in the middle with height Q . Now, we have satisfied the constraint of A , B and C . We have $N - A - B + C$ buildings remaining which should not be visible from either side. This can be done by assigning them height P and hiding them between the buildings with height Q .

- Case 2: $C = 1$. In this case, we cannot hide buildings between buildings with the maximum height since there is only one of them. We would have to look for some cases here:
 - $A + B - C = N$,
In this case, we have no buildings to hide, thus we can assign the buildings similar to Case 1 when $C > 1$ with heights P and Q.
 - Either $A > 1$ or $B > 1$. In this case, $A + B - C < N$. $A + B$ is at least 3 and $C = 1$. Thus, we can say that $N > 2$ holds. Consider any 3 values P, Q and R such that $1 \leq P < Q < R \leq N$. In this case, we can put $A - 1$ buildings with height Q on the left, $B - 1$ buildings with height Q on the right side and a building with height R in the middle. Now, we have satisfied the constraint of A , B and C . We have $N - A - B + C$ buildings remaining which should not be visible from either side. Let the height of remaining buildings be P. We have already placed at least 2 buildings, and all the buildings that we placed so far are higher than the buildings that we want to hide, so we can hide remaining buildings anywhere in between the already placed buildings.
 - $A = 1$ and $B = 1$
This means that the building with maximum height is both on the leftmost point and rightmost point and it is not possible for $N > 1$. So, the answer is IMPOSSIBLE in this case.

Checking which case our solution falls under takes constant time. We can then just assign the heights to the buildings in linear time. Hence, the complexity of the solution is $O(N)$.

Analysis: Longest Arithmetic

Consider a subarray of length K which is an arithmetic subarray, and let the elements of arithmetic subarray be $B_1, B_2, B_3, \dots, B_K$. We can say that $B_2 - B_1 = B_{i+1} - B_i$ for $1 \leq i < K$, because consecutive elements of arithmetic sequence should have a common difference.

Claim 1: In the given array, consider a subarray starting at index i and ending at index j . Now if this subarray is not arithmetic, there exists some index x such that $i \leq x < j$ and $A_{x+1} - A_x \neq A_{i+1} - A_i$. All subarrays starting at index i and ending at index y such that $x < y \leq N$, are not arithmetic because all such subarrays would contain index x such that $A_{x+1} - A_x \neq A_{i+1} - A_i$.

Test Set 1

For each element i such that $(1 \leq i < N)$, we consider each subarray starting at index i . Consider subarray(i, j) and start with $j = i$. Increment j while subarray(i, j) is an arithmetic subarray. For a fixed index i , we do not need to increment j after we find the first index such that subarray(i, j) is not an arithmetic subarray. None of the subarrays with i as a starting point and ending point after the index j will be arithmetic subarrays according to Claim 1. Let the maximum j for index i such that subarray(i, j) is an arithmetic subarray be max_j . We can conclude our approach as follows. Initialise the answer as 0. For each index i , find max_j . Update the answer if $\text{max_j} - i + 1$ is greater than the answer. For each index i , we would traverse $O(N)$ elements. Hence, the overall complexity of the solution is $O(N^2)$.

Sample Code (C++)

```
int maxArithmeticSubarray(vector<int> array) {
    int maxLen = 0;
    for(int i = 0; i < array.size() - 1; i++) {
        int j = i;
        int common_difference = array[i+1] - array[i];
        while(j < array.size() - 1 && (array[j + 1] - array[j] == common_difference))
            j++;
        int max_j = j;
        maxLen = max(maxLen, max_j - i + 1);
    }
    return maxLen;
}
```

Test Set 2

Consider an index i . Now consider all the subarrays (i, j) starting at index i and ending at index j . Start with $j = i$. Let the maximum index j where the subarray(i, j) is an arithmetic subarray be $j = x$. Let $A_{i+1} - A_i = D$. We can say that $A_{y+1} - A_y = D$ for all $i \leq y < x$. We have 2 cases now.

- Case 1: $x = N$,
In this case, subarray(i, N) is an arithmetic subarray. All subarrays(p, N) such that $i < p \leq N$, will have shorter length than subarray(i, N). Hence, we can discard all subarrays starting with index p .
- Case 2: $x \neq N$,
 $A_{x+1} - A_x \neq D$. We have already proved that we need not consider $j > x$ for index i as those subarrays will not be arithmetic using Claim 1. Now consider subarrays ($k, x + 1$) such that $(i + 1 \leq k < x)$. All these subarrays are not arithmetic because $A_{x+1} - A_x \neq D$ whereas $A_{k+1} - A_k = D$. Hence, we can discard all the subarray starting with index k . So, we can now shift the starting index to x .

We can conclude our approach as follows. Initialise the answer as 0. We maintain two pointers, left pointer i and right pointer j . For an index i , we try to find the longest arithmetic subarray starting at index i by incrementing j . Let the maximum j for index i such that subarray(i, j) is an arithmetic subarray be max_j . Update answer if $\text{max_j} - i + 1$ is greater than current answer. And then we shift our left pointer i to the current

max_j. We can see that both the pointers visit each element at most once. Hence, the complexity of the solution is $O(N)$.

Sample Code (C++)

```
int maxArithmeticSubarray(vector<int> array) {
    int maxLen = 0;
    for(int i = 0; i < array.size() - 1; i) {
        int j = i;
        int common_difference = array[i+1] - array[i];
        while(j < array.size() - 1 && (array[j + 1] - array[j] == common_difference))
            j++;
        int max_j = j;
        maxLen = max(maxLen, max_j - i + 1);
        i = max(i + 1, j);
    }
    return maxLen;
}
```

Analysis: Toys

Let us say Axel can play with the toys in multiple rounds (starting from 1), where each round starts with the toy that has the smallest number and finishes with the toy that has the largest number. There are a few observations:

1. Axel can play with each toy at least once, in round 1.
2. If Axel cannot play indefinitely, in round 2 he will get stuck at some toy and start crying.
3. If Axel can play with a toy more than twice (i.e. reaches round 3), he can keep playing indefinitely.

Test Set 1

For this test set, we can generate all subsets of the toys in order. For each subset, we can loop over the toys in the subset and calculate the maximum time Axel can play. This maximum time will be either INDEFINITELY (if Axel can play more than twice as in Observation 3), or sum of the enjoyments of the toys played until Axel gets stuck as in Observation 2. We can keep track of the maximum time for each subset and if two subsets have the same maximum time, we will consider the one with minimum toys removed (maximum subset size). As number of subsets will be 2^N and for each subset, time spent by Axel is calculated in linear time, total complexity of the solution will be $O(N \times 2^N)$.

Test Set 2

Let us say we have K toys. If Axel can play indefinitely with these toys, for each toy, its remembrance should be less than or equal to the sum of enjoyment of all other toys except this one. If SUM is total sum of enjoyments of all the toys that Axel can play with, we can say, for each $i = 1$ to K , $R_i \leq SUM - E_i$ Or $R_i + E_i \leq SUM$.

If Axel cannot play with these K toys indefinitely, we can try removing all the toys violating the condition $R_i + E_i \leq SUM$, so that we can get a list of toys with no violation and Axel can play indefinitely. Here, we can remove the toys violating the condition in any order. But for simplicity, we will first remove a toy for which $R_i + E_i$ is the largest. The reason being, if we remove some other toy, it will only decrease the SUM by enjoyment of the other toy and this toy would still be violating the condition $R_i + E_i \leq SUM$.

In this test set, we can keep a list of the toys such that Axel can play with them indefinitely. Also, we will have a track of total time played till now (let us say *cur_time*), maximum possible time that Axel can spend playing (let us say *max_time*) and count of toys removed. Initially, *cur_time* will be total time taken in round 1 i.e. SUM as per Observation 1. Now, we will simulate the round 2 and have an empty list initially. We will keep adding toys 1 to N one by one to this list. After adding a toy to the list, we will check if we have some toy in the list violating the condition $R_i + E_i \leq SUM$. If so, we will remove the toy violating the condition that has the largest $R_i + E_i$. Once we add a toy to the list, we will add its enjoyment to *cur_time* and while removing a toy from the list, we will remove its corresponding time (i.e. twice the enjoyment time for this toy) from *cur_time*, and remove its enjoyment time from the SUM . After each toy is added and toys violating the condition are removed, we will update *max_time* if *cur_time* is optimal. Also, we will keep track of total toys removed from the list and update that as well while updating *max_time*. Finally, when all the toys have been added to the list and processed, if the list is not empty, Axel can play with the toys indefinitely. Else, we have *max_time* and corresponding toys removed as our answer.

To maintain the list of toys and remove a toy with the largest $R_i + E_i$ efficiently, we can use [Priority Queue](#) to store toys in decreasing order of their $R_i + E_i$. As we are processing each toy twice, once for calculating initial *SUM*, and other for processing second round where in priority queue each toy can be added/removed maximum one time, total complexity of solution will be $O(N \log N)$.